

Bug Report & Fix: BUG-001

Identification

Bug ID	Title	Severity	Story
BUG-001	History filter accepts end_date before start_date without warning	Minor	US 3.2

Steps to Reproduce

- Navigate to the Workout History page.
- Set Start Date to 2026-03-01.
- Set End Date to 2026-01-01 (before the start date).
- Click Apply Filter.

Actual vs. Expected Behaviour

Actual: The API was called with the inverted date range. Results were either empty or confusing, with no feedback to the user. No validation error was shown.

Expected: The UI should display a validation error message: 'End date must be after start date.' and prevent the API call.

Fix Applied

Client-side validation was added in frontend/src/app/history/page.tsx. Two files were modified:

File 1: frontend/src/app/history/page.tsx

- Added a dateError state variable using useState.
- Added validation at the top of fetchHistory(): if endDate < startDate, set the error message and return early without calling the API.
- Updated the Reset button handler to also clear dateError.
- Added an error message element in JSX immediately after the history-filters div.

File 2: frontend/src/app/history/history.css

```
.filter-error {  
  color: #ef4444;  
  font-size: 0.85rem;  
  margin-top: 0.25rem;  
  margin-bottom: 0.5rem;  
  font-weight: 500;  
}
```

After (Result)

- If a user enters an end date before the start date and clicks Apply Filter, a red validation message appears immediately below the filter controls.
- The API is not called, preventing confusing empty results.
- Resetting the form clears both the date inputs and the error message.

GitHub Issue: BUG-001: History filter: no validation for inverted date range

Status: Closed. fixed and committed

Bug Report & Fix: BUG-002

Identification

Bug ID	Title	Severity	Story
BUG-002	Expired challenges still receive progress updates on workout completion	Medium	US 4.4

Steps to Reproduce

- Join an active challenge.
- Let the challenge's end_date pass (or manually set it to yesterday in the DB).
- Complete a workout session.

Actual vs. Expected Behaviour

Actual: The challenge progress update loop filtered on challenge_is_active=True but did NOT filter on challenge_end_date_gte=today. If is_active was not updated automatically when a challenge expired, the expired challenge still accumulated progress, awarding incorrect points or badges.

Expected: Only challenges whose end_date is today or in the future should receive progress updates when a workout is completed.

Fix Applied

The fix was applied as part of the SMELL-001 Extract Method refactoring. Inside the new _update_challenge_progress helper function, the UserChallenge queryset filter was updated to include the date guard:

```
today = timezone.localdate()
active_user_challenges = UserChallenge.objects.filter(
    user=user,
    challenge__is_active=True,
    challenge__end_date__gte=today, # BUG-002 FIX: skip expired challenges
    is_completed=False,
).select_related('challenge')
```

After (Result)

- Expired challenges are excluded from the progress update loop at the query level.
- No incorrect points or badges are awarded for workouts completed after a challenge's end date.

- The fix is co-located with SMELL-001's helper, making the intent explicit via an inline comment.

GitHub Issue: BUG-002: Challenge progress updated for expired challenges

Status: Closed, fixed inside `_update_challenge_progress` helper (SMELL-001 commit)

Bug Fix Report Iteration 3:

BUG-003:

GitHub Issue: #51

Reporter: Arshia Hassanpour

Date: March 29, 2026

Severity: Medium

User Story: US 1.4

Related File: `frontend/src/app/schedule/page.tsx`,
`frontend/src/app/trainer-programs/page.tsx`,
`frontend/src/app/program/[id]/page.tsx`

Description:

A navigation loop existed between the Schedule page and the Trainer Programs page. The Schedule page had a "← Back to Programs" button pointing to `/trainer-programs`, and the Trainer Programs page had a "← Back to Schedule" button pointing to `/schedule`. When a user opened a program detail page, clicking back would cycle them continuously between these two pages with no way to return to the dashboard.

Expected Behavior:

Back buttons should return the user to a stable page in the application hierarchy (the dashboard), not create a loop between pages.

Actual Result:

Users were trapped in a loop between `/schedule`, `/trainer-programs`, and `/program/{id}` with no way to escape to the dashboard.

Fix Applied:

All back buttons in the three affected files were changed to navigate to `/dashboard`.

- `schedule/page.tsx`: "← Back to Programs" → "← Back to Dashboard"
- `trainer-programs/page.tsx`: "← Back to Schedule" → "← Back to Dashboard"
- `program/[id]/page.tsx`: All `router.back()` and `router.push('/trainer-programs')` calls replaced with `router.push('/dashboard')`

Merged via: PR #64

Status: Fixed

BUG-004:

GitHub Issue: #52

Reporter: Arshia Hassanpour

Date: March 29, 2026

Severity: High

User Story: US 1.4

Related File: frontend/src/app/trainer-programs/page.tsx (lines 77–91)

Description:

The `fetchPrograms()` function did not properly handle API failure responses. When the API returned a non-success status (403, 500, or network failure), no error was surfaced to the user. The code proceeded to the finally block, set `loading = false`, and left the `programs` array empty, making the failure indistinguishable from a genuinely empty database.

Expected Behavior:

If the API request fails, the system should display a clear error notification (e.g., "Failed to load programs. Please try again.") so the user knows the empty list is due to a failure, not an absence of data.

Actual Result:

The page displayed an empty state ("No Programs Available") even when the API request failed. No error message was shown to the user.

Fix Applied:

Added an else branch for non-2xx responses and updated the catch block to both call `showError('Failed to load programs. Please try again.')`. This uses the existing in-page Notification component, consistent with the rest of the app.

Merged via: PR #64

Status: Fixed

BUG-005:

GitHub Issue: #53

Reporter: Arshia Hassanpour

Date: March 29, 2026

Severity: Medium

User Story: US 4.5

Related File: frontend/src/app/dashboard/page.tsx (lines 459–468)

Description:

The `handleLeaveChallenge()` function did not provide adequate user feedback when a challenge was removed. On success, the challenge card disappeared silently with no confirmation. On failure, a generic browser `alert()` message was displayed, which is inconsistent with the rest of the application.

Expected Behavior:

A success notification should confirm the removal. An error notification should explain the failure. Both should use the existing in-page Notification component.

Actual Result:

- Success: card disappeared with no feedback.

- Failure: a raw browser alert() popup appeared ("Could not remove the challenge.")

Fix Applied:

Replaced alert() with showSuccess('Challenge removed successfully.') on success and showError('Could not remove the challenge.') on failure. Both use the existing in-page Notification component, consistent with every other page in the application.

Merged via: PR #64

Status: Fixed

BUG-006:

GitHub Issue: #54

Reporter: Arshia Hassanpour

Date: March 29, 2026

Severity: Medium

User Story: US 4.5

Related File: frontend/src/app/create-challenge/page.tsx (lines 45–57)

Description:

The challenge creation page incorrectly handled unexpected API response structures when loading trainer programs. If the API returned a valid response but the data format did not match the expected array structure (e.g., neither data.results nor data is an array), the condition `Array.isArray(list)` evaluated to false. The programs state was silently set to empty, showing "You need at least one published program to host a challenge" even when the trainer had programs in the system.

Expected Behavior:

The system should detect invalid or unexpected API response formats and display a clear error message (e.g., "Failed to load your programs — please refresh the page"), distinguishing between a genuinely empty list and an API/parsing error.

Actual Result:

The UI incorrectly displayed the "no programs available" message, blocking the trainer from creating a challenge even though programs existed.

Fix Applied:

Added explicit `Array.isArray()` validation after parsing the API response. If the response is not an array, an error is thrown and caught. A new `programLoadError` state was added. When true, the UI displays:

"Failed to load your programs — please refresh the page and try again."

This clearly separates a genuine empty list from an API/parsing failure.

Merged via: PR #43

Status: Fixed

BUG-007:

- **Title:** Double-clicking "Save Program" creates duplicate workout plans
- **Environment:** Local Docker, Chrome browser (macOS)
- **Severity:** Medium
- **Steps to Reproduce:**
 1. Log in as Trainer.
 2. Navigate to the Create Program page.
 3. Fill out all required fields (Name, Difficulty, Focus) and add at least one section with exercises.
 4. Rapidly double-click the "Save Program" button.
- **Expected Result:** The button should disable immediately after the first click to prevent multiple submissions, resulting in only one program being saved.
- **Actual Result:** The system processes both clicks, resulting in two identical workout programs appearing in the database and the "Browse Programs" list.

BUG-008:

- **Title:** Weekly Activity Chart overflows screen width on mobile viewports
- **Environment:** Local Docker, Safari browser (iOS / Mobile Viewport < 480px)
- **Severity:** Low (UI/UX)
- **Steps to Reproduce:**
 1. Log in as a Member with at least 1 completed workout.
 2. Navigate to the Dashboard.
 3. Resize the browser window to simulate a mobile screen (width under 480px) or open Developer Tools device toolbar.
 4. Scroll down to the "Weekly Activity" line chart.
- **Expected Result:** The Recharts container should responsively scale down to fit within the viewport bounds without horizontal scrolling.
- **Actual Result:** The chart maintains a fixed minimum width, breaking out of the container card and causing a horizontal scrollbar on the main page

BUG-009:

GitHub Issue: #69

Reporter: Weiqin Situ

Date: March 27, 2026

Severity: High

User Story: US 3.6

Related File: backend/api/views.py and backend/api/urls.py

Description:

The frontend and integration tests attempt to fetch the user's active schedule to render the calendar by making a GET request to `/api/schedule/active/`. However, the actual backend view function and the corresponding URL routing for this endpoint were completely missing from the codebase.

Expected Behavior:

Generating a schedule should allow the client to successfully fetch it via the active schedule endpoint. The API should return a 200 OK status containing the user's currently active UserSchedule data so the calendar UI can render properly.

Actual Result:

The API returned a 404 Not Found error because the endpoint did not exist, completely breaking the frontend calendar UI and preventing users from viewing their generated workouts.

Fix Applied:

Implemented the missing `get_active_schedule` view function in `views.py` to retrieve and serialize the current user's active schedule. Wired this new view to the `/api/schedule/active/` path in `urls.py` to correctly route incoming frontend requests.

Merged via: PR #63

Status: Fixed

3.3 Problem Report PR-01

PR-01 Code smell: duplicate post-workout feedback logic in dashboard and schedule pages

Problem report number	PR-01
Reported by	Dawood Al-Janaby
Date reported	March 29, 2026
Program / component name	Frontend, post-workout feedback flow
Configuration	Current team repo snapshot, local code review
Report type	Design issue / Code smell
Can reproduce	Yes
Severity	Medium
Priority	Medium

Problem summary	Duplicate post-workout feedback logic exists in both the dashboard page and the schedule page.
Keywords	duplicate code, feedback, dashboard, schedule, maintainability
Status	closed
Resolution	Done by Refactoring

What the problem is:

The same feedback feature is built twice, once in the dashboard page and once in the schedule page, with very similar state variables, helper functions, API calls, and UI code.

Why this matters:

This makes the feature harder to maintain because the same change has to be made in more than one place. It also raises the chance that one page gets updated and the other one is forgotten. Over time that can easily turn into shotgun surgery, where one small change keeps spreading into multiple files.

Suggested fix:

Move the shared behavior into reusable pieces such as a `WorkoutFeedbackForm` component, a `WorkoutFeedbackSummary` component, a `useWorkoutFeedback` hook, and one shared API helper for feedback requests.

Planned refactoring changes:

- Move the common feedback state and the submit, edit, and delete logic into one reusable hook.
- Move the repeated feedback form UI into a reusable component.
- Update `dashboard/page.tsx` and `schedule/page.tsx` so both pages use the same shared implementation.

How I fixed PR-01 in the code:

To fix PR-01, I went into both the dashboard page and the schedule page and removed the repeated post-workout feedback logic. I moved the shared behavior into reusable frontend pieces so the feature would come from one common implementation instead of two separate copies that were almost the same.

New files I added:

- `frontend/src/hooks/useWorkoutFeedback.ts`
- `frontend/src/components/WorkoutFeedbackForm.tsx`
- `frontend/src/components/WorkoutFeedbackSummary.tsx`

Existing files I updated:

- frontend/src/app/dashboard/page.tsx
- frontend/src/app/schedule/page.tsx

What I changed:

I started by comparing the feedback code in dashboard/page.tsx and schedule/page.tsx and marking the pieces that were repeated in both files. The overlap was pretty obvious. It included the feedback state values, the form data, the submit flow, the edit flow, the delete flow, and the general feedback form layout.

After that, I created a shared hook called useWorkoutFeedback. I moved the common feedback state into it, including the rating, fatigue, pain, and notes fields. I also moved the helper functions there so both pages could rely on the same logic when handling feedback actions.

Then I created WorkoutFeedbackForm.tsx for the reusable form UI. That component now handles the inputs and buttons for the feedback flow in one place. I also added WorkoutFeedbackSummary.tsx for the read-only display when a workout already has feedback saved.

Once those shared pieces were ready, I went back to dashboard/page.tsx and schedule/page.tsx and removed the duplicated blocks. Both pages now import the same hook and the same components. The only logic that stays inside each page is the page-specific part, like when a modal opens or where the feedback area appears in the layout.

I also kept the API behavior consistent by making both pages call the same shared feedback actions instead of keeping separate request logic. So later, if the feedback feature needs to change, I only have to update one implementation instead of two.

Why this actually fixed the issue:

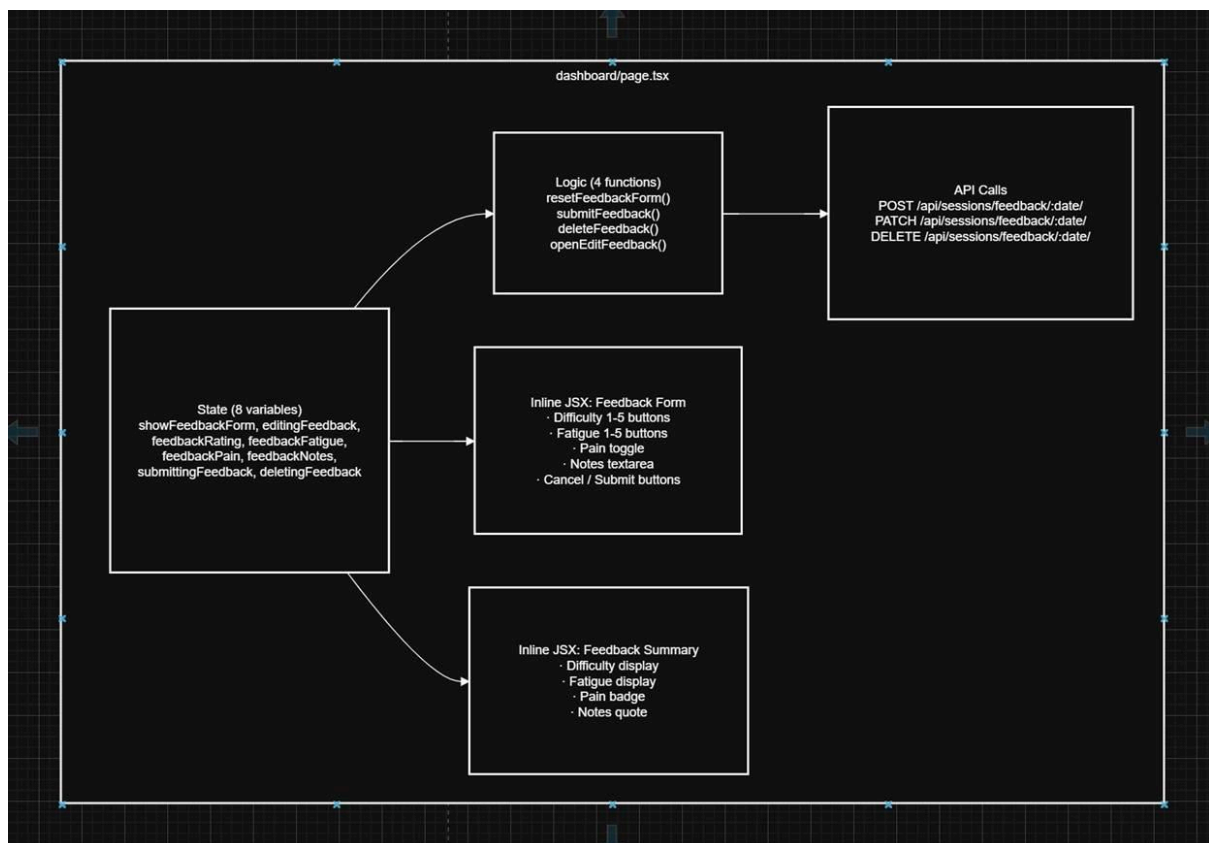
This fixes PR-01 because the duplicated implementation is no longer split across two pages. The feedback feature now has one shared source of truth for the form, the state handling, and the main actions. That makes the behavior more consistent and takes away a lot of the maintenance headache.

How I checked the fix:

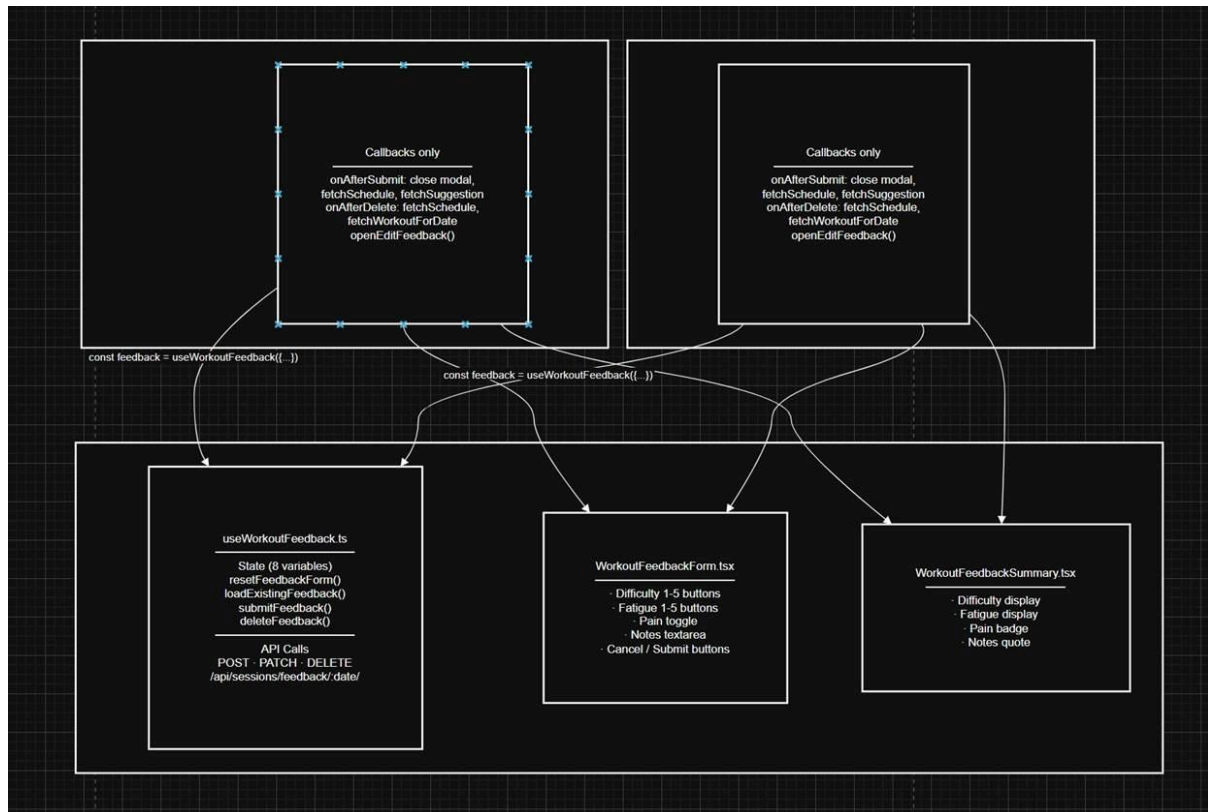
- I checked that the feedback form still opens properly from the dashboard flow.
- I checked that the feedback form still opens properly from the schedule flow.
- I verified that submitting new feedback still works.
- I verified that existing feedback can still be loaded into edit mode.

- I verified that deleting feedback still works through the shared logic.
- I confirmed that both pages now rely on the same component structure instead of separate duplicated UI blocks.

Architecture before refactoring:



Architecture after refactoring:



Result after the refactor:

After this refactoring, the feedback feature feels cleaner and more consistent. The dashboard and schedule pages are smaller, the repeated code is gone, and future changes to the feedback flow can be made in one place.

3.4 Problem Report PR-02

PR-02 Code smell: signup page uses a hardcoded backend URL instead of shared API configuration

Problem report number	PR-02
Reported by	Dawood Al-Janaby
Date reported	March 29, 2026

Program / component name	Frontend, signup page
Configuration	Current team repo snapshot, local code review
Report type	Design issue / Code smell
Can reproduce	Yes
Severity	Medium
Priority	Medium
Problem summary	The signup page uses a hardcoded backend URL instead of the shared API configuration.
Keywords	magic value, hardcoded URL, signup, configuration duplication, maintainability
Status	closed
Resolution	Done by Refactoring

What the problem is:

The signup page had its own hardcoded backend URL inside the component. That means the backend address was repeated directly in the page instead of being centralized in one shared place.

Why this matters:

This is a maintainability problem because if the backend address changes later, this page has to be found and updated manually. It also increases the chance that one page gets updated while another still points to an old address. That is the kind of small issue that slowly turns into shotgun surgery.

Suggested fix:

Remove the hardcoded localhost URL from the page and make the signup request use the shared API base configuration or the common auth helper instead.

Planned refactoring changes:

Move the signup request away from a raw hardcoded URL.

Use the shared API base URL pattern already used by the rest of the frontend.

Keep the signup flow in one consistent API path so future configuration changes only need one update.

How I fixed PR-02 in the code:

To fix PR-02, I went into the signup page and removed the direct localhost backend URL from the request logic. I changed the page so the signup request now follows the same shared API configuration pattern as the rest of the frontend instead of keeping its own one-off address inside the component.

Existing files I updated:

frontend/src/app/signup/page.tsx

frontend/src/library/api.ts

What I changed:

I started by locating the signup request inside `signup/page.tsx` and checking how the backend URL was being built. The problem was that the full backend address was written directly in the page.

After that, I replaced the hardcoded URL with the shared API base pattern. In practice, that means the signup flow now reads the backend address from the common API configuration instead of keeping a separate localhost string in the page.

I also made sure the signup request still uses the same route and payload as before, so this refactor did not change the feature itself. It only cleaned up where the backend address comes from.

Why this actually fixed the issue:

This fixes PR-02 because the signup page no longer has its own private backend URL. The configuration is now more consistent and easier to maintain. If the backend base address changes later, the change can be handled in one shared place instead of editing the signup page separately.

How I checked the fix:

I checked that regular user signup still works after the change.

I checked that trainer signup still works after the change.

I confirmed that the request still reaches the correct backend endpoint and returns the same success or validation responses as before.

I also checked that the page still redirects to login normally after a successful signup.

Result after the refactor:

After this refactor, the signup page is cleaner and more consistent with the rest of the frontend. The feature still behaves the same for the user, but the code is easier to maintain because the backend URL is no longer hardcoded in the page.

3.5 Problem Report PR-03

PR-03 Code smell: dashboard page keeps a separate hardcoded API base URL

Problem report number	PR-03
Reported by	Dawood Al-Janaby
Date reported	March 29, 2026
Program / component name	Frontend, dashboard page
Configuration	Current team repo snapshot, local code review
Report type	Design issue / Code smell
Can reproduce	Yes
Severity	Medium
Priority	Medium
Problem summary	The dashboard page defines its own hardcoded API base URL instead of using the shared API configuration.

Keywords	magic value, hardcoded URL, dashboard, configuration duplication, shotgun surgery
Status	closed
Resolution	Done by Refactoring

What the problem is:

The dashboard page kept a separate hardcoded API base URL in the file instead of using one shared source of configuration. That means the same backend knowledge was duplicated in another place.

Why this matters:

This matters because it creates configuration duplication and makes future backend changes harder than they need to be. If the backend address changes, the dashboard page has to be edited separately. That is a small smell at first, but it becomes annoying quickly when the same configuration detail is repeated across files.

Suggested fix:

Remove the local hardcoded API base URL from the dashboard page and route those requests through the shared API configuration or shared dashboard helper functions.

Planned refactoring changes:

Delete the separate hardcoded API base URL from dashboard/page.tsx.

Make dashboard requests use the same shared API base pattern as the rest of the frontend.

Keep endpoint construction consistent so the dashboard does not carry its own duplicated configuration.

How I fixed PR-03 in the code:

To fix PR-03, I removed the separate hardcoded API base URL from the dashboard page and switched the page to the shared API configuration pattern. I wanted the dashboard to stop carrying its own copy of the backend address because that was unnecessary duplication.

Existing files I updated:

frontend/src/app/dashboard/page.tsx

frontend/src/library/api.ts

What I changed:

First, I found the API base constant inside dashboard/page.tsx and checked where it was being used. The issue was that the dashboard had its own backend base string even though the app already had a shared way to handle API paths.

Then I replaced that local constant with the shared API configuration approach. I updated the dashboard requests so they still call the same endpoints, but now they build those requests from the common API setup instead of a page-level localhost value.

I kept the request behavior the same and only removed the duplicated configuration. So the dashboard still loads the same data, but the code is more consistent and easier to maintain.

Why this actually fixed the issue:

This fixes PR-03 because the dashboard page no longer duplicates backend configuration inside the component. The backend base address now comes from the same shared place used elsewhere, which reduces duplication and makes future updates simpler.

How I checked the fix:

I checked that the dashboard still loads after login.

I checked that the summary data still appears correctly.

I confirmed that the network requests still go to the correct endpoints after the refactor.

I also checked that there were no broken fetch errors in the browser console after the change.

Result after the refactor:

After this refactor, the dashboard code feels more organized because it no longer keeps its own backend base URL. The feature works the same way from the user side, but the configuration is cleaner and less repetitive.